

Week 11: Cloud ETL with Prefect

Python 200 I Mentor Session

What This Week Covers

Students have all the pieces of the pipeline. This week wires them together.

- **Prefect refresher** — `@task`, `@flow`, the UI
- **What changes in cloud ETL** — why retries, logging, and idempotency matter more with real I/O
- **Building the pipeline** — four tasks wrapped in a single `@flow`
- **Production patterns** — retries, `raise_for_status`, structured logging, idempotency
- **Reading the Prefect UI** — interpreting task states, finding failure traces

This is the capstone week. The new content is the production patterns — the pipeline structure itself applies everything from weeks 1, 4, 9, and 10.

Prefect Refresher

Students used Prefect in Week 1 for a local pipeline. The core API is the same.

Two building blocks:

- `@task` — a single unit of work: call an API, write to a database, run a classifier
- `@flow` — the orchestrator: calls tasks in order, tracks state, captures logs

```
from prefect import flow, task

@task
def extract() -> list: ...

@task
def load_raw(records: list) -> None: ...

@flow(log_prints=True)
def etl_pipeline():
    raw_records = extract()
    load_raw(raw_records)

if __name__ == "__main__":
    etl_pipeline()
```

`log_prints=True` captures all `print()` output as structured Prefect log entries.

What Changes in Cloud ETL

In Week 1, tasks did local work. If they failed, re-running was free and harmless.

In a cloud ETL pipeline, each task does real I/O with real consequences:

- **Extract** makes an HTTP call that can fail due to rate limits or network timeouts
- **Transform** calls the OpenAI API for each record — each call costs money; mid-run failures waste both time and money
- **Load** writes to Supabase — a failed run after partial inserts may leave the database in an inconsistent state

Failures are not just inconvenient. Incorrect data can silently flow downstream. Observability — knowing what ran, what failed, and why — matters much more in a cloud pipeline than in a local script.

Discussion: In Week 1, if the pipeline crashed halfway through, what was the cost? In Week 11, what are the costs of the same crash?

The ETL Skeleton

Four tasks. Two load steps. The flow structure makes the pipeline stages visible at a glance.

```
@task
def extract() -> list:
    ... # Open-Meteo API → list of raw weather records

@task
def load_raw(records: list) -> None:
    ... # Upsert raw records into weather_raw

@task
def transform(raw_records: list) -> list:
    ... # ML classify + LLM enrich → list of enrichment records

@task
def load_enriched(enrichment_records: list) -> None:
    ... # Upsert enrichment records into weather_enriched

@flow(log_prints=True)
def etl_pipeline():
    raw_records = extract()
    load_raw(raw_records)
    enrichment_records = transform(raw_records)
    load_enriched(enrichment_records)
```

One task, one responsibility. Data dependencies are visible from the function signatures.

Lesson 1 Demo: Prefect Revisited

Open: Lesson 1 → Prefect revisited

Focus on	Skim for now
@task and @flow — core API is identical to Week 1	Prefect 2.x vs 3.x UI differences (core API is the same)
The ETL skeleton — four tasks, clear data flow	Prefect Cloud and deployments (mentioned in Lesson 3)
The Prefect UI — start the server, identify task states, open the logs tab	get_run_logger() syntax (covered in Lesson 3)

Start the Prefect server (`prefect server start`) and open `localhost:4200` . Walk through a completed run from Week 1 together — click into tasks, review log output.

Discussion: The skeleton uses `extract() → load_raw() → transform() → load_enriched()` . Why does `load_raw` come before `transform` , even though `transform` reads from `raw_records` (not from the database)?

Production Pattern: Retries

External API calls fail sometimes — rate limits, network timeouts, momentary unavailability. These are not bugs in your code, but they will crash the pipeline without retries.

```
@task(retries=2, retry_delay_seconds=10)
def extract() -> list:
    url = "https://archive-api.open-meteo.com/v1/archive"
    response = requests.get(url, params=params)
    response.raise_for_status()
    ...
```

When the task raises an exception, Prefect waits 10 seconds and tries again, up to 2 more times. Each attempt is logged separately.

Retries are for transient external failures — not programming bugs.

If your code has a logic error, retrying delays the inevitable and makes debugging noisier. Two or three retries with a 10–30 second delay is the common production default.

Production Pattern: Error Handling

`raise_for_status()` is a small habit with a large payoff.

```
# Silent failure – pipeline continues with bad data
response = requests.get(url)
if response.status_code != 200:
    print("Something went wrong")

# Loud failure – Prefect marks the task Failed; downstream tasks don't run
response = requests.get(url)
response.raise_for_status()
```

Without `raise_for_status()`, a 404 or 500 response is treated as success. The script tries to parse the error body as weather data, fails with a confusing `KeyError`, and may write garbage to the database.

General principle: in data pipelines, visible failures are safer than silent corruption. A failed run is inconvenient; incorrect data flowing through a system is much harder to detect and fix later.

Production Pattern: Logging

`log_prints=True` on the flow captures all `print()` output automatically. For most pipelines, that's enough:

```
print(f"Upserted {len(records)} rows into weather_raw")
```

For severity levels, use `get_run_logger()` inside a task:

```
from prefect.logging import get_run_logger

@task
def load_enriched(enrichment_records: list) -> None:
    logger = get_run_logger()
    logger.info(f"Upserting {len(enrichment_records)} records")
    if not enrichment_records:
        logger.warning("No enrichment records – transform may have found nothing to process")
    return
...
```

`logger.info()` — routine progress. `logger.warning()` — unusual but non-fatal. `logger.error()` — serious problem.

Good logging is the first thing engineers look at when something breaks overnight.

Lesson 2 Demo: Building the Pipeline

Open: Lesson 2 → building the pipeline

Focus on	Skim for now
The four task implementations — each wraps code from weeks 9 and 10	Clients initialized outside tasks (closure behavior)
<code>retries=2, retry_delay_seconds=10</code> on <code>extract</code> and <code>load</code> tasks	Progress prints inside the <code>transform</code> task
The <code>load_enriched</code> guard — <code>if not enrichment_records: return</code>	<code>log_prints=True</code> on the flow

Run the complete pipeline end-to-end. Watch the Prefect UI as each task transitions through `Running` → `Completed`. Open a task's logs and find the print output.

Discussion: Where would you add retries in the `transform` task — on the entire task, on the LLM loop, or both? What are the tradeoffs?

Production Pattern: Idempotency

A pipeline is **idempotent** if running it multiple times produces the same result as running it once.

Upsert with `on_conflict` — both load tasks use it. Re-running updates existing rows in place; the database always ends up in a consistent state.

The incremental check — the transform task checks `weather_enriched` before classifying:

```
already_done = {r["date"] for r in supabase.table("weather_enriched").select("date").execute().data}
to_process   = [r for r in raw_records if r["date"] not in already_done]
```

If the pipeline crashes during the LLM step after processing 200 of 365 records, re-running picks up from record 201. The 200 already-written records are detected and skipped. No manual cleanup, no duplicate rows, no extra LLM API costs.

Discussion: Why check `already_done` against `weather_enriched`, not against `weather_raw`? What would happen if you used the wrong table?

Reading the Prefect UI

Open `http://localhost:4200` and click into the `etl-pipeline` run.

- **Green (Completed)** — task succeeded; click to see its logs
- **Red (Failed)** — task raised an uncaught exception; click the *Logs* tab to find the traceback
- **Blue (Running)** — task is currently executing

Tasks that completed before a failure remain green — you can see exactly where in the pipeline the failure occurred without re-running the whole thing.

A useful exercise: intentionally cause a failure and observe. Remove `SUPABASE_KEY` from `.env`, run the pipeline, read the failure trace in the UI. Restore the key and re-run — the incremental check means the pipeline picks up where it left off.

Lesson 3 Demo: Production-Ready Patterns

Open: Lesson 3 → making pipelines production-ready

Focus on	Skim for now
Retries vs programming bugs — when to use each	Prefect Cloud and hosted deployments
<code>raise_for_status()</code> — loud failure vs silent corruption	Scheduled deployments and Prefect workers
Idempotency — upsert + incremental check together	Batching and concurrency for large-scale LLM calls

Intentionally break the pipeline (wrong key, bad URL) and read the failure in the UI. Then fix it and re-run — confirm the incremental check picks up where it left off.

Discussion: The lesson says the jump from "run locally" to "scheduled deployment" is smaller than it looks. What would need to change in the code to run this pipeline on a schedule?

Week 11 Wrap-Up

- **Four tasks** — `extract`, `load_raw`, `transform`, `load_enriched`; one responsibility each; clear data dependencies
- **Retries** — for transient external failures; not for bugs; two or three attempts with 10–30 second delays is the production default
- `raise_for_status()` — fail loudly and early; bad data flowing silently downstream is worse than a failed run
- **Logging** — `log_prints=True` for simple pipelines; `get_run_logger()` for severity levels; good logs are how you debug overnight failures
- **Idempotency** — upsert + incremental check together; re-running after any failure is always safe

Takeaway: You built a cloud ETL pipeline from scratch — Week 1 Prefect + Week 4 model persistence + Week 9 database writes + Week 10 double-transform, all working together in a realistic workflow. The production patterns here — retries, loud failures, idempotency — are the same patterns used in real data engineering pipelines at scale.